

TÉCNICAS DE HACKING

Práctica 2: Reconocimiento Activo

[Marcos Pérez García] Fecha de entrega: 25 de mayo de 2026

Índice

1. Resumen	3
2. Introducción	4
3. Desarrollo	4
3.1. Descubrimiento de hosts con Scapy	4
3.1.1. Metodología	4
3.1.2. Entorno de pruebas	4
3.2. Comportamiento por defecto de Nmap y estados de puerto	5
3.2.1. Estados de puerto: definición, estímulos y respuestas	5
3.2.2. Comportamiento por defecto de Nmap	5
4. Resultados	7
4.1. Descubrimiento de hosts con Scapy	7
4.1.1. Prueba 1: Host activo individual	7
4.1.2. Prueba 2: IP sin host activo	7
4.1.3. Prueba 3: Rango completo de red	7
4.2. Comportamiento por defecto de Nmap: evidencias con Tcpdump	7
4.2.1. Metodología de captura	7
4.2.2. Resultados del análisis de tráfico	7
4.2.3. Escaneo a target-multi (172.20.0.11)	8
4.2.4. Escaneo a target-icmp-only (172.20.0.13)	8
4.2.5. Verificación de puerto cerrado	8
5. Conclusiones	9
6. Bibliografía	10

1. Resumen

Este documento presenta los resultados de la Práctica 2 de Reconocimiento Activo, correspondiente a la asignatura de Técnicas de Hacking. El objetivo principal es implementar y analizar técnicas de descubrimiento de hosts mediante el envío de estímulos de red y la observación de sus respuestas, así como comprender el comportamiento por defecto de la herramienta Nmap en el reconocimiento de puertos.

Para la parte práctica se ha desarrollado una herramienta en Python utilizando la biblioteca Scapy, capaz de construir y enviar paquetes ICMP (Timestamp Request), TCP (ACK) y UDP para identificar hosts activos en una red. Las pruebas se han realizado sobre un entorno controlado con Docker, simulando una red con múltiples servicios expuestos.

Adicionalmente, se ha analizado el tráfico generado por Nmap mediante el sniffer Tcpdump, identificando el número exacto de paquetes enviados (2005), los puertos escaneados por defecto (1000) y los estímulos TCP SYN utilizados para determinar el estado de cada puerto, así como las respuestas SYN+ACK y RST que permiten clasificarlos como abiertos o cerrados.

Los resultados confirman que la herramienta desarrollada detecta correctamente hosts activos e inactivos mediante los tres protocolos, y que Nmap, en su configuración por defecto, escanea 1000 puertos TCP enviando paquetes SYN y analizando las respuestas para determinar el estado de los puertos.

2. Introducción

El reconocimiento activo es una fase fundamental en cualquier auditoría de seguridad o prueba de penetración. Consiste en interactuar directamente con los sistemas objetivo para obtener información sobre su estado, servicios expuestos y configuración de red. A diferencia del reconocimiento pasivo, donde el atacante no interactúa con el objetivo, el reconocimiento activo implica el envío de paquetes y la interpretación de las respuestas obtenidas.

En esta práctica se abordan dos aspectos esenciales del reconocimiento activo:

1. Descubrimiento de hosts: identificar qué dispositivos se encuentran activos en una red mediante el envío de estímulos a nivel de red (ICMP) y transporte (TCP, UDP), utilizando la biblioteca Scapy de Python.
2. Comportamiento por defecto de Nmap: analizar en profundidad el funcionamiento de Nmap cuando se ejecuta sin opciones adicionales, comprendiendo los paquetes que envía, los puertos que escanea y cómo determina el estado de cada puerto mediante el análisis de las respuestas.

El entorno de pruebas se ha implementado utilizando contenedores Docker, lo que permite simular una red aislada con diferentes configuraciones de servicios (HTTP, SSH, UDP) y hosts con distintos niveles de respuesta.

3. Desarrollo

3.1. Descubrimiento de hosts con Scapy

3.1.1. Metodología

Para el descubrimiento de hosts se ha implementado la función `texttt{craft_discovery_pkts}` en Python, utilizando la biblioteca Scapy. Esta función construye paquetes de red personalizados según los protocolos especificados:

- `textit{ICMP Timestamp Request (tipo 13):}` solicita al host destino su marca de tiempo actual. Si el host está activo y no filtra este tipo de mensajes ICMP, responderá con un ICMP Timestamp Reply (tipo 14), según lo definido en el RFC 792.
- `textit{TCP ACK:}` envía un paquete TCP con el flag ACK activado. Según el RFC 793, un host que recibe un paquete ACK no solicitado debe responder con un paquete RST (Reset), lo que confirma su presencia en la red. Este comportamiento es independiente de si el puerto está abierto o cerrado.
- `textit{UDP:}` envía un datagrama UDP a un puerto específico. Si el puerto está cerrado, el host debería responder con un mensaje ICMP Port Unreachable (tipo 3, código 3). Si el puerto está abierto, generalmente no se recibe respuesta, según el RFC 768.

3.1.2. Entorno de pruebas

Se ha configurado un entorno controlado mediante Docker, con la siguiente topología de red:

Contenedor	Dirección IP	Servicios
target-web	172.20.0.10	HTTP (puerto 80)
target-multi	172.20.0.11	SSH (puerto 22), HTTP (puerto 8080)
target-udp	172.20.0.12	UDP (puerto 53)
target-icmp-only	172.20.0.13	Ninguno (solo responde ICMP)
attacker	172.20.0.2	Kali Linux con Scapy y Nmap

Table 1: Topología de la red de pruebas

3.2. Comportamiento por defecto de Nmap y estados de puerto

3.2.1. Estados de puerto: definición, estímulos y respuestas

Nmap clasifica los puertos en seis estados posibles. La determinación del estado se realiza mediante el envío de paquetes TCP SYN y el análisis de las respuestas recibidas, prestando atención a los campos específicos de cada paquete. A continuación se detallan los tres estados principales:

v(0.5em)

Abierto (open): una aplicación en el host destino está aceptando conexiones en ese puerto. El estímulo enviado es un paquete TCP con el flag SYN activado (SYN=1). Si el puerto está abierto, el destino responde con un paquete TCP que tiene los flags SYN y ACK activados (SYN=1, ACK=1), completando la segunda fase del three-way handshake. Nmap interpreta esta respuesta y marca el puerto como abierto, tras lo cual envía un paquete RST para no completar la conexión (en escaneo SYN stealth).

v(0.5em)

Cerrado (closed): el puerto es accesible pero no hay ninguna aplicación escuchando en él. El estímulo es el mismo: un paquete TCP SYN. Sin embargo, al no existir un servicio vinculado al puerto, el sistema operativo del host destino responde con un paquete TCP con el flag RST activado (RST=1, ACK=1). Esta respuesta indica que el puerto está disponible a nivel de red pero no hay servicio asociado.

v(0.5em)

Filtrado (filtered): un firewall, filtro de paquetes u otro dispositivo de red impide que la sonda llegue al puerto destino. Como resultado, no se recibe ninguna respuesta, o bien se recibe un mensaje ICMP tipo 3 (Destination Unreachable) con códigos 1, 2, 3, 9, 10 o 13. Nmap no puede determinar si el puerto está abierto o cerrado porque la respuesta ha sido bloqueada.

v(0.5em)

Estado	Estímulo	Respuesta	Campos de paquete analizados
Abierto	TCP SYN	TCP SYN+ACK	Flags: SYN=1, ACK=1
Cerrado	TCP SYN	TCP RST	Flags: RST=1, ACK=1
Filtrado	TCP SYN	Sin respuesta / ICMP tipo 3	ICMP type=3, code=1/2/3/9/10/13

Table 2: Determinación del estado de puertos según estímulos y respuestas analizadas

v(1em)

Esta clasificación es fundamental para el auditor, ya que permite identificar no solo los servicios disponibles, sino también la presencia de firewalls y la postura de seguridad del objetivo.

3.2.2. Comportamiento por defecto de Nmap

Cuando se ejecuta Nmap sin opciones adicionales (simplemente `nmap`), se activan los siguientes comportamientos preconfigurados:

- Realiza un escaneo de tipo TCP SYN (conocido como SYN stealth o half-open scan) si se ejecuta con privilegios de root. Esto implica enviar paquetes TCP con flag SYN y analizar las respuestas sin completar el three-way handshake.
- Escanea los 1000 puertos TCP más comunes, definidos en el archivo `nmap-services`. Esta selección cubre los puertos donde estadísticamente se encuentran la mayoría de servicios.

- Por cada puerto, envía un paquete TCP SYN desde un puerto origen aleatorio. Si recibe SYN+ACK, marca el puerto como abierto y envía RST para cerrar. Si recibe RST, marca como cerrado. Si no recibe respuesta tras varias retransmisiones, marca como filtrado.
- Previamente al escaneo de puertos, Nmap realiza un host discovery enviando ICMP Echo Request y TCP SYN al puerto 443 para confirmar que el host está activo.

4. Resultados

4.1. Descubrimiento de hosts con Scapy

4.1.1. Prueba 1: Host activo individual

Se ejecutó la herramienta contra la IP 172.20.0.10 (target-web), obteniendo los siguientes resultados:

Protocolo	Respuesta obtenida	Conclusión
ICMP (type 13)	ICMP Timestamp Reply (type 14)	Host activo
TCP (ACK puerto 80)	TCP RST	Host activo
UDP (puerto 80)	Respuesta ICMP tipo 3 código 3	Host activo

Table 3: Resultados del escaneo a 172.20.0.10 con los tres protocolos

Los tres protocolos confirmaron que el host está activo. La respuesta ICMP type 14 confirma que el host soporta la solicitud de timestamp sin filtrarla. El flag RST en TCP indica que el host recibió el paquete ACK no solicitado y respondió según el RFC 793. La respuesta ICMP Port Unreachable (tipo 3, código 3) ante el paquete UDP confirma que el puerto 80/UDP está cerrado pero el host es accesible.

4.1.2. Prueba 2: IP sin host activo

Se escaneó la IP 172.20.0.99, que no corresponde a ningún contenedor en la red:

Protocolo	Respuesta	Conclusión
ICMP	Sin respuesta	Host inactivo
TCP	Sin respuesta	Host inactivo

Table 4: Resultados del escaneo a 172.20.0.99

La ausencia total de respuestas confirma que no hay un host activo en esa dirección IP. Scapy mostró el aviso `texttt{MAC address to reach destination not found. Using broadcast}`, indicando que no se pudo resolver la dirección MAC del destino mediante ARP, lo cual es consistente con una IP no asignada en la red local.

4.1.3. Prueba 3: Rango completo de red

Se escaneó el rango 172.20.0.0/24, detectando correctamente los hosts activos configurados en el entorno Docker.

4.2. Comportamiento por defecto de Nmap: evidencias con Tcpdump

4.2.1. Metodología de captura

Para analizar en profundidad el comportamiento de Nmap, se utilizó Tcpdump como sniffer de paquetes. Se capturó todo el tráfico entre el contenedor atacante (172.20.0.2) y el objetivo target-web (172.20.0.10) durante un escaneo por defecto. La captura se realizó con el comando:

```
v(0.5em) texttt{tcpdump -i br-XXXXXX -w captura_nmap.pcap} v(0.5em)
```

Posteriormente, se aplicó un filtro para aislar únicamente el tráfico relevante:

```
v(0.5em) texttt{tcpdump -r captura_nmap.pcap -nn host 172.20.0.10 and host 172.20.0.2} v(0.5em)
```

4.2.2. Resultados del análisis de tráfico

El análisis de la captura reveló el siguiente patrón de tráfico:

- El atacante (172.20.0.2) envía paquetes TCP SYN desde un puerto origen (36556) hacia los 1000 puertos destino de 172.20.0.10.

- Para cada puerto cerrado, el objetivo responde con TCP RST, indicando que el puerto no tiene servicio.
- Para el puerto 80 (HTTP), se observa una respuesta diferente (SYN+ACK), lo que llevó a Nmap a marcarlo como abierto.
- Tras recibir SYN+ACK en el puerto 80, el atacante envía RST para no completar la conexión.

Métrica	Valor obtenido
Total de paquetes capturados (filtrados)	2005
Puertos TCP escaneados	1000
Tipo de escaneo detectado	TCP SYN (half-open)
Puertos abiertos detectados	1 (puerto 80/tcp HTTP)
Puertos cerrados	999
Flags en paquetes enviados	SYN
Flags en respuestas (puertos cerrados)	RST, ACK
Flags en respuesta (puerto abierto)	SYN, ACK

Table 5: Análisis detallado del tráfico capturado con Tcpdump durante el escaneo de Nmap a 172.20.0.10 v(1em)

Cada puerto cerrado generó 2 paquetes: un SYN saliente y un RST entrante (999 puertos $\times$ 2 = 1998 paquetes). El puerto 80 abierto generó 3 paquetes: SYN, SYN+ACK y RST de cierre. Los paquetes restantes corresponden a tráfico ARP para resolución de direcciones MAC.

4.2.3. Escaneo a target-multi (172.20.0.11)

La salida de Nmap mostró el puerto 22/tcp (SSH) como abierto. El puerto 8080 no aparece en el escaneo por defecto al no estar entre los 1000 puertos más comunes, lo que demuestra la limitación del escaneo estándar.

4.2.4. Escaneo a target-icmp-only (172.20.0.13)

Los 1000 puertos escaneados aparecen como cerrados. El host responde a ICMP (lo que permite a Nmap determinar que está activo) pero no tiene servicios TCP disponibles, demostrando que Nmap utiliza previamente un mecanismo de host discovery basado en ICMP antes del escaneo de puertos.

4.2.5. Verificación de puerto cerrado

Se verificó explícitamente el puerto 22 en target-web con `texttt{nmap -p 22 172.20.0.10}`, confirmando su estado cerrado (TCP RST). Esto valida que el host solo expone el servicio HTTP en el puerto 80.

5. Conclusiones

El desarrollo de esta práctica ha permitido alcanzar los siguientes objetivos y conclusiones:

1. Descubrimiento de hosts multicapa: la combinación de protocolos ICMP, TCP y UDP permite una detección robusta de hosts activos, ya que distintas configuraciones de firewall pueden bloquear unos protocolos pero no otros. La herramienta implementada con Scapy demostró ser eficaz en la identificación tanto de hosts activos como inactivos.
2. Determinación del estado de puertos: el análisis de los campos de los paquetes TCP (flags SYN, ACK, RST) y de los mensajes ICMP (type y code) es el mecanismo fundamental mediante el cual Nmap clasifica los puertos como abiertos, cerrados o filtrados. Comprender estos estímulos y respuestas es esencial para interpretar correctamente los resultados de un escaneo.
3. Comportamiento de Nmap evidenciado con Tcpdump: la captura de tráfico permitió verificar que el escaneo por defecto de Nmap utiliza TCP SYN sobre los 1000 puertos más comunes, generando 2005 paquetes en la prueba realizada. El sniffer confirmó el patrón SYN/RST para puertos cerrados y SYN/SYN+ACK/RST para puertos abiertos.
4. Importancia del análisis de tráfico: el uso de sniffers como Tcpdump no solo permite validar el funcionamiento de las herramientas, sino que constituye una evidencia objetiva e incuestionable del comportamiento real de la red, más allá de lo que muestran las herramientas en su salida estándar.
5. Entornos controlados: la virtualización con Docker proporciona un entorno seguro y reproducible para la experimentación con técnicas de reconocimiento activo, evitando riesgos legales y éticos asociados al escaneo de redes no autorizadas.

6. Bibliografia

- Gordon Lyon. textit{Nmap Network Scanning}. 2009. URL: <https://nmap.org/book/>
- Philippe Biondi. textit{Scapy: Packet crafting for Python}. 2025. URL: <https://scapy.readthedocs.io/>
- Docker Inc. textit{Docker Documentation}. 2025. URL: <https://docs.docker.com/>
- The Tcpdump Group. textit{TCPDUMP & LIBPCAP}. 2025. URL: <https://www.tcpdump.org/>
- Chris McNab. textit{Network Security Assessment: Know Your Network}. 3.^a ed. O'Reilly Media, 2016.
- Jon Postel. textit{RFC 792: Internet Control Message Protocol}. 1981.
- Jon Postel. textit{RFC 793: Transmission Control Protocol}. 1981.
- Jon Postel. textit{RFC 768: User Datagram Protocol}. 1980.